Getting Started with C#

Soyeb Ghachi

Technical Trainer | Mentor |SME

Overview

- Introduction to C#
- Evolution of C#
- Keyword in C#
- Data Types in C#
- Dynamic Data Type
- var Type
- Value Types and Reference Types
- What is Variable
- Region in C#
- Operators in C#
- Constants in C#
- Read only in C#
- Type conversion in C#
- Boxing and Un-Boxing
- Coding Standards

Introduction to C#.Net

- C# (pronounced as ‘C Sharp’) it’s "simple, modern, general-purpose object-oriented language."
- It is modern language that combines the power of C++ with productivity of VB and elegance of Java.
- C# is a modern, type safe programming language, object oriented language that enables programmers to quickly and easily build solutions for the Microsoft .NET platform.
- **Anders Hejlsberg** introduced C# for .Net Framework 1.0. and was approved by the European Computer Manufacturers Association (ECMA) and International Standards Organization (ISO).

Evolution to C#.Net

Year	C# Version	.NET Platform / SDK	Major Features Introduced
2002	C# 1.0	.NET Framework 1.0	First release, classes, structs, interfaces, delegates, events, properties, basic OOP features
2003	C# 1.2	.NET Framework 1.1	Minor updates, improved security and interoperability
2005	C# 2.0	.NET Framework 2.0	Generics, Nullable types, Anonymous methods, Iterators (yield), Partial classes
2007	C# 3.0	.NET Framework 3.5	LINQ, Lambda expressions, Extension methods, Anonymous types, Object & Collection initializers
2010	C# 4.0	.NET Framework 4.0	Dynamic binding (dynamic keyword), Named & Optional parameters, COM interoperability improvements
2012	C# 5.0	.NET Framework 4.5	Async and Await for asynchronous programming
2015	C# 6.0	.NET Framework 4.6 / .NET Core 1.0	Expression-bodied members, Null conditional operator (?.), String interpolation, Auto-property initializers
2017	C# 7.0	.NET Core 2.0	Tuples, Pattern matching, Local functions, Ref returns, Out variables

2018	C# 7.3	.NET Core 2.1	Stackalloc improvements, unmanaged constraint, performance improvements
2019	C# 8.0	.NET Core 3.0	Nullable reference types, Default interface methods, Async streams, Switch expressions
2020	C# 9.0	.NET 5	Records, Init-only properties, Pattern matching improvements, Top-level statements
2021	C# 10	.NET 6 (LTS)	Global using directives, File-scoped namespaces, Record structs, Improved lambda expressions
2022	C# 11	.NET 7	Required members, Raw string literals, Generic math support, List patterns
2023	C# 12	.NET 8 (LTS)	Primary constructors, Collection expressions, Default lambda parameters, Inline arrays
2024	C# 13	.NET 9	Improved params collections, New lock object support, Better pattern matching
2025+	C# 14 (Preview)	.NET 10	Planned improvements in pattern matching, performance enhancements, better memory safety

Keywords in C#

Contains rich set of 76 reserved keywords. Keywords can be used as identifiers prefaced by an @.

Keywords	Keywords	Keywords	Keywords
abstract	as	base	bool
Break	Byte	Case	Catch
Char	Checked	Class	Const
Continue	Decimal	Default	Delegate
Do	Double	Else	Enum
Event	Explicit	Extern	False
Finally	Fixed	Float	For
Foreach	Goto	If	Implicit
In	Int	Interface	Internal
Is	Lock	Long	Namespace
New	Null	Object	Operator
Out	Override	Params	Private
Protected	Public	Readonly	Ref

Keywords	Keywords	Keywords	Keywords
True	Try	Typeof	Uint
Ulong	Unchecked	Unsafe	Catch
Char	Checked	Class	Const
Using	Virtual	Void	While

Value Type and Reference Type

Types in C#

C# supports two kinds of types: Value Types and Reference Types.

Types	Description	Types	Description
Value Types	Includes simple data types such as int, char, bool, enums	Value Types	Includes simple data types such as int, char, bool, enums
Reference Types	Includes object, class, interface, delegate, and array types	Reference Types	Includes object, class, interface, delegate, and array types

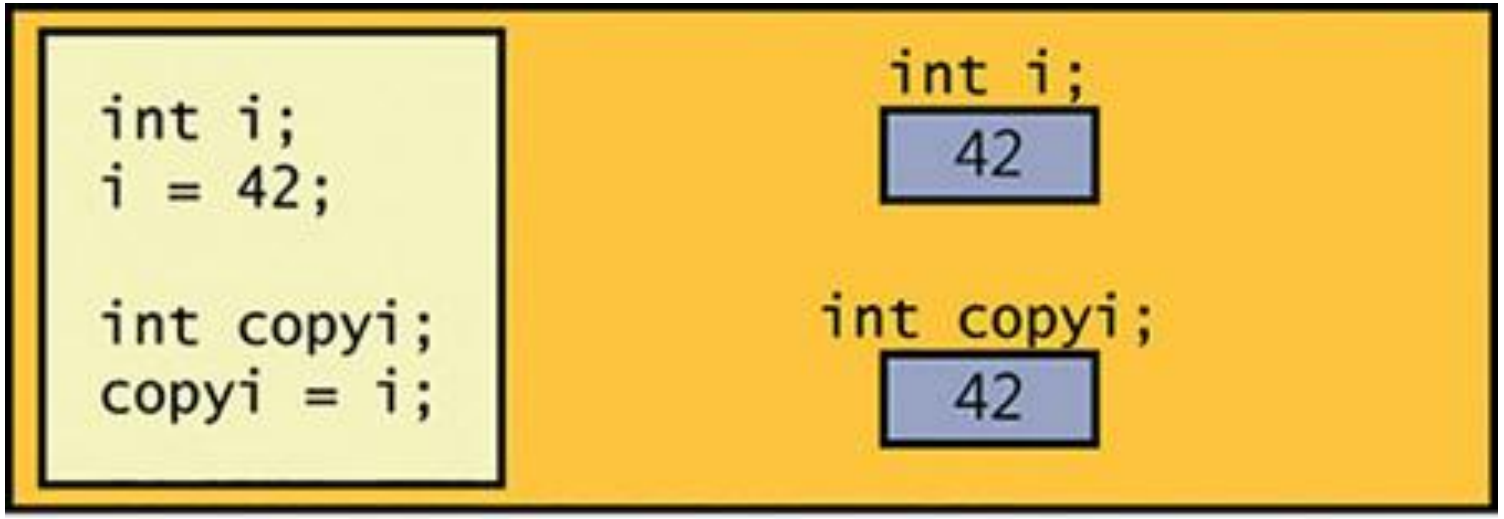
- **Value Types-** Value type objects direct contain the actual data in a variables. With value types, the variables each have their own copy of the data, and it is not possible for operations on one to affect the other.

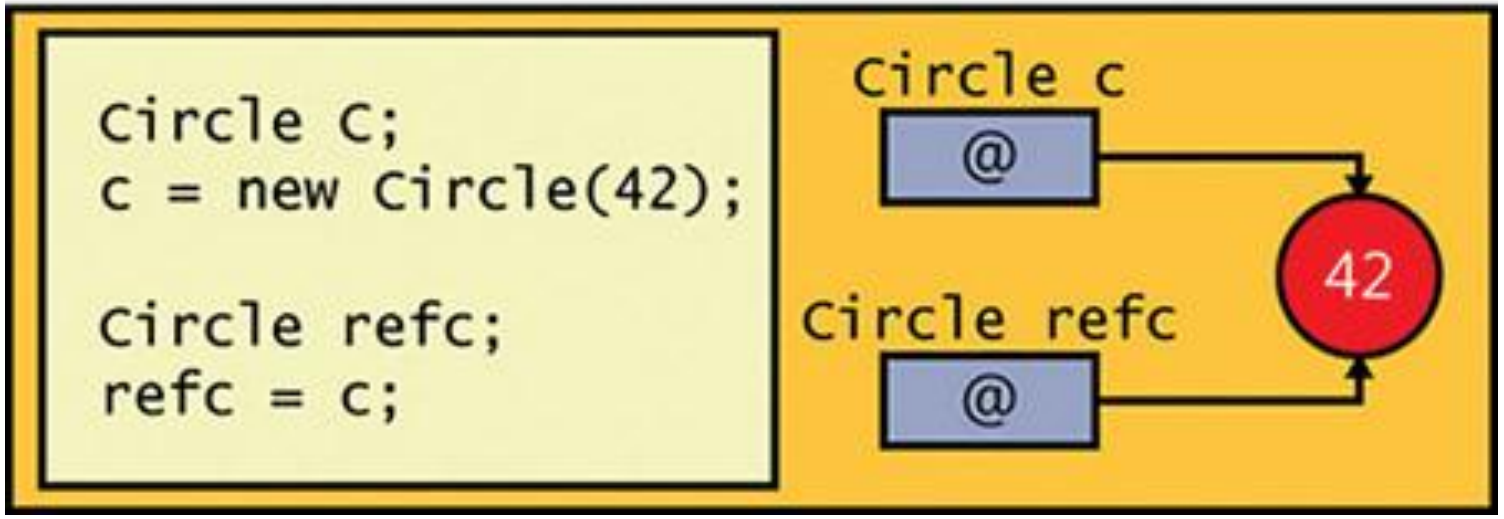
```
int i = 10;
```

- **Reference Types-** Reference type variables stores the reference of the actual data. With reference types, it is possible for two variables to reference the same object, and thus possible for operations on one variable to affect the object referenced by the other variable.

```
MyClass cls1 = new MyClass();
```

Value Type	Reference Type
All simple types are value types.	All Complex Types are Ref. type except struct
Allocate memory at Compile Time.	Allocate memory at runtime.
Uses stack to store direct value.	Uses memory heap (block of memory) to store memory address rather than value.
Value type is more faster than Ref. type.	Ref. type is portable.
Creates a complete copy of the data	Copies only the memory address (reference)
e.g. int, char, bool, enum, struct. etc.	e.g. class, object (System.Object), array, string, interface, delegate etc


int i;
i = 42;
int copyi;
copyi = i;
int i;
42
int copyi;
42

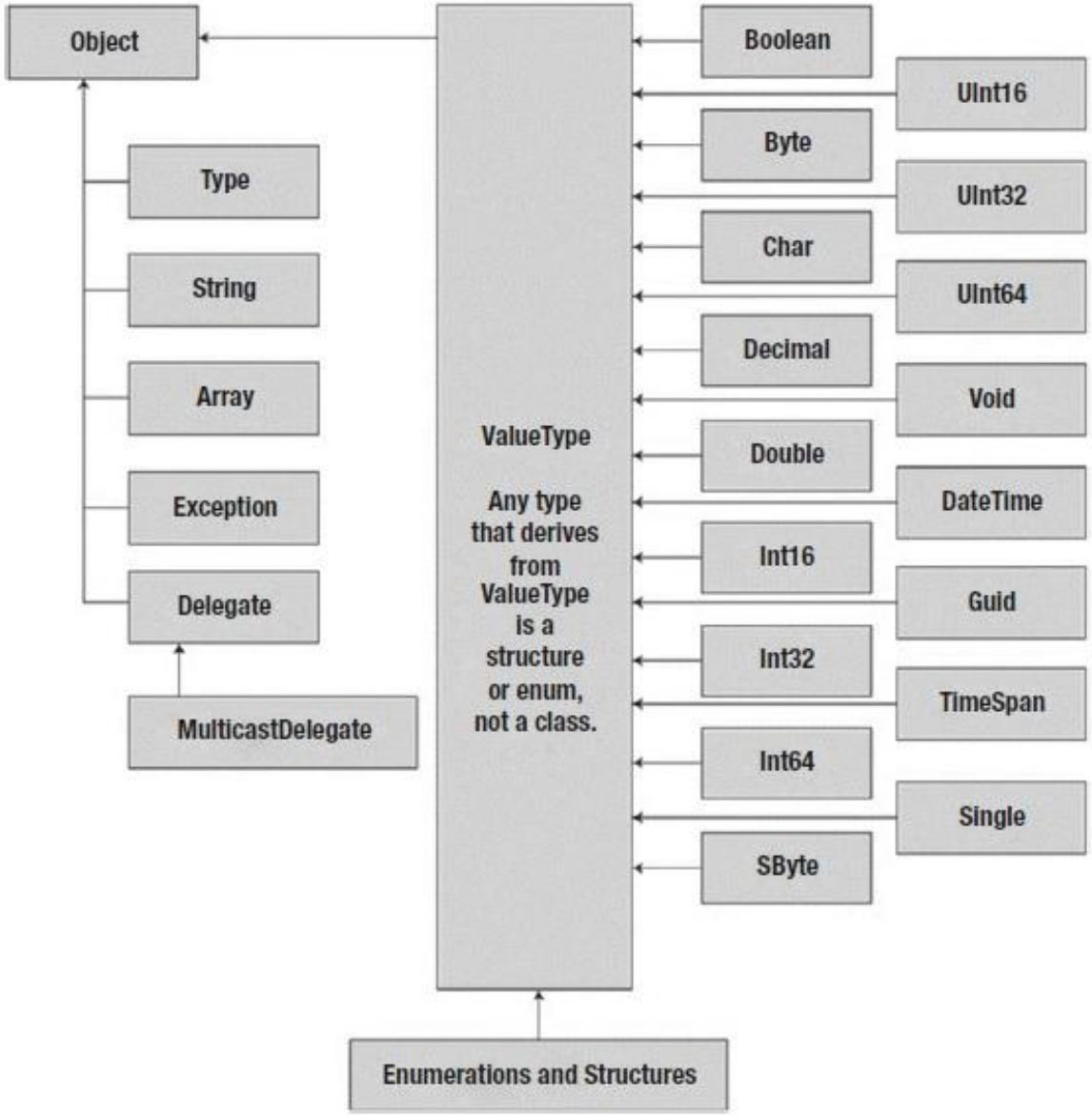

Circle c;
c = new Circle(42);
Circle refc;
refc = c;
Circle c
@
Circle refc
@
42

Data Types in C#

Most of the data type in C# are taken from C and C++. This tables lists data types, their description, and a sample example.

Type	Description	Example
object	The base type of all types	object obj = null;
string	String type - sequence of Unicode characters	string str = "Mahesh";
bool	Boolean type; a bool value is either true or false	bool val1 = true; bool val2 = false;
char	Character type; a char value is a Unicode character	char val = 'h';

Hierarchy of System.Object



Numerical Data Types

TYPE	DESCRIPTION	MINIMUM VALUE	MAXIMUM VALUE
byte	Unsigned byte	0	255
sbyte	Signed byte	-128	127
short	Signed byte	-32 768	32 767
ushort	Unsigned byte	0	65 535
int	Signed byte	-2 147 483 648	2 147 483 647
uint	Unsigned byte	0	4 294 967 295
long	Signed long	$-9x10^{8}$	$9x10^{8}$
ulong	Unsigned long	0	$1,8x10^{19}$

Non-integer Data Types

All non-integer data types are signed.

Arithmetic Overflow

Overflow is an operation that occurs when a calculation produces a result that is greater in magnitude than that which a given register or storage location can store or represent.

TYPE	DESCRIPTION	PRECISION
float	Single Precision Number	7 digits
double	Double Precision Number	15 or 16 digits
decimal	Decimal Number	28 or 29 digits

Dynamic Data Type

- **Dynamic Data Type**

The dynamic keyword brings exciting new features to C# 4.

Dynamic Type means that you can store any type of value in the dynamic data type variable because type checking for dynamic types of variables takes place at run-time.

```
dynamic dynInt = 100;
dynamic dynStr = "Hello";
```

Var Type

var Type:

•var type is special data type to store variant type of data like an Object type.

•We can store variant type of data into var as well as we can store multiple values of different type.

```
static void Main(string[] args)
        {
            var name = "Welcome";
            var a = 34;
            Console.WriteLine("name={0} a={1}",name,a);
            Console.ReadLine();
        }
```

Using of C# var keyword in LINQ

```
static void Main(string[] args)
        {
            int[] array = { 1, 2, 4, 6, 8, 9, 11, 14, 15 };

            var a = from i in array where i % 2 == 0 select i;

            foreach(var _a in a)
            {
                Console.WriteLine("{0}\n",_a);
            }
            Console.Read();
        }
```

Differences between var and dynamic type

var	dynamic
It is introduced in C# 3.0.	It is introduced in C# 4.0
statically typed.	dynamically typed.
The type of the variable is decided by the compiler at compile time.	The type of the variable is decided by the compiler at run time.
Must be initialized.	The variable of this type need not be initialized Because the compiler does not know the type of the variable at compile time.
If the variable does not initialized it throw an error.	If the variable does not initialized it will not throw an error.
It support intelliSense in visual studio.	It does not support intelliSense in visual studio

var myvalue = 10; // statement 1
myvalue = “Hello World”; // error
	dynamic myvalue = 10; // statement 1
	myvalue = “Hello World”; // statement 2

It cannot be used for properties or returning values from the function. It can only used as a local variable in function.	It can be used for properties or returning values from the function

What is Variable

What is Variable?

Variable is place holder for values in memory or simply variable is object of data Type.

How can we Declare and assign a value to the Variable?

- Declaration:

```
<type> <variable name>
int      a;
string   name;
```

- Assignment/Initialization:

```
A=10;
Name="XYZ";
```

Region in C#

Region in C#

Console Based program

```
using  System;
using  System.Collections;
#region entry point
public  class  EntryClass
{
        public  static  void Main()
        {
            NewClass t = new NewClass();
        }
}
#endregion entry point
```

Sample C# program-Console Based program

```
using System;
using System.Collections.Generic;
using System.Text;
namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            System.Console.WriteLine("HELLO WORLD");
            System.Console.ReadLine();
        }

    }
}
```

OUTPUT:

HELLO WORLD

Operators in C#

- **Arithmetic Operators:**

Operator	Function	Example
+	addition	11 + 2
-	subtraction	11 - 2
*	multiplication	11 * 2
/	division	11 / 2
%	remainder	11 % 2
++	Increment by 1	++expr1; expr2++;
--	Decrement by 1	--expr1; expr2--;

- **Relational Operators**

Operator	Function	Example
<	Less than	expr1 < expr2;
>	Greater than	expr1 > expr2;
<=	Less than or equal to	expr1 <= expr2;
>=	Greater than or equal to	expr1 >= expr2;
==	Equality	expr1 == expr2;
!=	Inequality	expr1 != expr2;

- **Conditional Operators**

Operator	Function	Example
!	Logical NOT	Dept!="Hr."
||	Logical OR (short circuit)	percentage>60 ||experience>2;
&&	Logical AND (short circuit)	Percentage>60 && experience>2;
?:	Ternary	Age>=18 ? "Major" : "Minor";

- typeof is used to find out the managed type of type at run time.

  Syntax: Type b1 = typeof(object);

  Both 'is' and 'as' keywords are used for type casting in C#.

- is operator is of boolean type whereas as operator is not of boolean type. The is operator returns true if the given object is of the same type whereas as operator returns the object when they are compatible with the given type.

  Syntax: a1 is A;

- as **operator is used to perform conversion between compatible reference types or Nullable types.** This operator returns the object when they are compatible with the given type and return null if the conversion is not possible instead of raising an exception.

  Syntax: string s = myObjects[0] as string;

Const and Read-Only

- **Constants :**

Classes and structs can declare constants as members.

Constants are values which are known at compile time and do not change. Constants are declared as a field, using the const keyword before the type of the field. Constants must be initialized as they are declared.

For example:

Multiple constants of the same type can be declared at the

same time, for example:

```
class Calendar2{
 static void Main(string[] args)
        {

            const int months = 12, weeks = 52, days = 365;
            const double daysPerWeek = days / weeks;
            const double daysPerMonth = days / months;

            Console.WriteLine(daysPerWeek);
            Console.WriteLine(daysPerMonth);
            Console.ReadLine();
        }
}
```

- **Read-Only :**

The readonly keyword in C# is a modifier that indicates a field, struct, or struct member can only be assigned a value during its declaration or within a constructor of the same class or struct. After the constructor finishes, the value cannot be changed.

When to use Readonly

We can use Readonly when its value is not an absolute constant, which means it can be changed frequently, such as dollars vs INR. In this requirement, we can set the value through a configuration file or another variable expression to avoid changing the class file frequently

```
class Person
{
    readonly int age;

    public Person(int a)
    {
        age = a;   // Allowed
    }

    public void Display()
    {
        Console.WriteLine(age);
    }
}
```

🔑 Major Differences

Feature	const	readonly
Assignment Time	Compile time	Runtime
Where value assigned	Only at declaration	Declaration or Constructor
Static	Always static	Not necessarily static
Memory	Stored in metadata	Stored in object instance
Can depend on runtime value	❌ No	✔ Yes
Per object value	❌ Same for all	✔ Can differ

Type Conversion

Type Conversion :

There are two types of type conversion implicit and explicit:

Implicit: Automatic compiler conversion where data loss is not an issue.

e.g. int iVal = 34;

long lVal = iVal;

Explicit: A conversion where data loss may happen and is recommended that the programmer writes additional processing

e.g. long lVal = 123456;

int iVal = (int) lVal;

C# Implicit Conversion:

```
using System;
using System.Collections.Generic;
using System.Text;
namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
            int x = 2;
            double y = 12.2;
            double z;
            z = x + y;     //x is automatically converted into a double
            Console.WriteLine(z);

            Console.Read();

        }
    }
}
```

C# Explicite Conversion:

```
using System;
using System.Collections.Generic;
using System.Text;
namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
           double x = 2.1;
            int y = 12;

             int z = (int)x + y;      //Explicit conversion from double to int

            Console.WriteLine(z);

            Console.Read();
        }
    }
}
```

C# Explicite Conversion using Convert class:

```
using System;
using System.Collections.Generic;
using System.Text;
namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {

            double lVal = 123.45;

            int iVal = Convert.ToInt32(lVal);        //double is converted to int

            Console.WriteLine(iVal);
            Console.Read();
        }
    }
}
```

C# Explicite Conversion using Convert class to take input from user:

```
using System;
using System.Collections.Generic;
using System.Text;
namespace ConsoleApplication1
{
    class Program
    {
        static void Main(string[] args)
        {
           int y;
            y = Convert.ToInt32(Console.ReadLine());

            Console.WriteLine(y);

            Console.Read();
        }
    }
}
```

int.Parse(), Convert.ToInt32() and int.TryParse()

	int.Parse()	Convert.ToInt32()	int.TryParse()
	int.Parse(string s)	Convert.ToInt32(string value)	int.TryParse (string s,out int result)
string str = "20";	20	20	20
string str = null;	ArgumentNullException	0	Failed to Convert but returns 0 and doesn't throw the exception
string str = “abc";	FormatException	FormatException	Failed to Convert but returns 0 and doesn't throw the exception
string str = “555555745455454 5454545";	OverflowException	OverflowException	Failed to Convert but returns 0 and doesn't throw the exception
bool b = false;	Not Supporting to convert	0	Failed to Convert but returns 0 and doesn't throw an Exception

```
string val =null;
int value = int.Parse(val);
```

ArgumentNullException

```
string val = "100.11";
int value = int.Parse(val);
```

FormatException

```
string val ="99999999999999999";
int value = int.Parse(val);
```

OverflowException

```
string val = null;
int result;
bool ifSuccess = int.TryParse(val, out result);
```

ifSuccess = false | result = 0

```
string val = "100.11";
int result;
bool ifSuccess = int.TryParse(val, out result);
```

ifSuccess = false | result = 0

```
string val = "999999999999999";
int result;
bool ifSuccess = int.TryParse(val, out result);
```

ifSuccess = false | result = 0

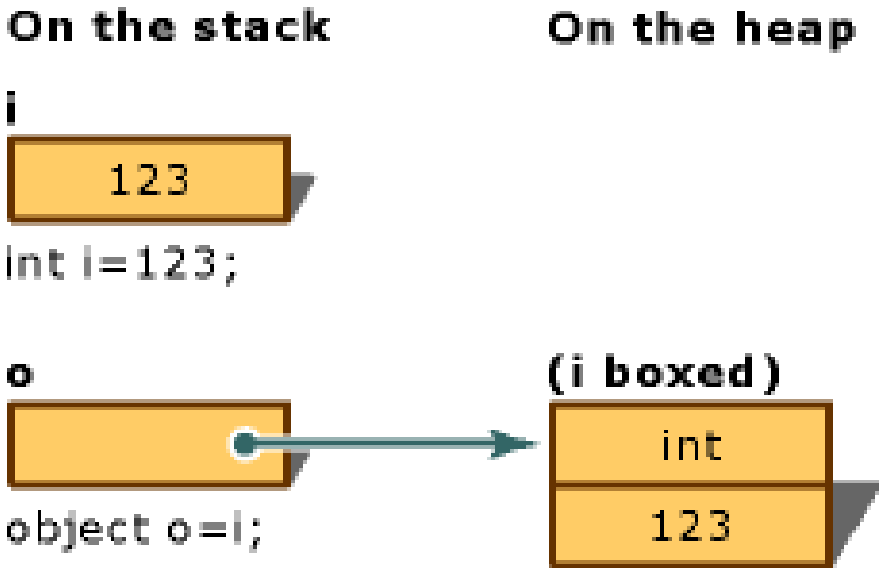
- **Boxing and Un-Boxing**

Object (System.Object) is the ultimate base class for all types.

Any type can be upcast to object.System.Object is such an important class that C# provides the object keyword as an alias for System.Object.

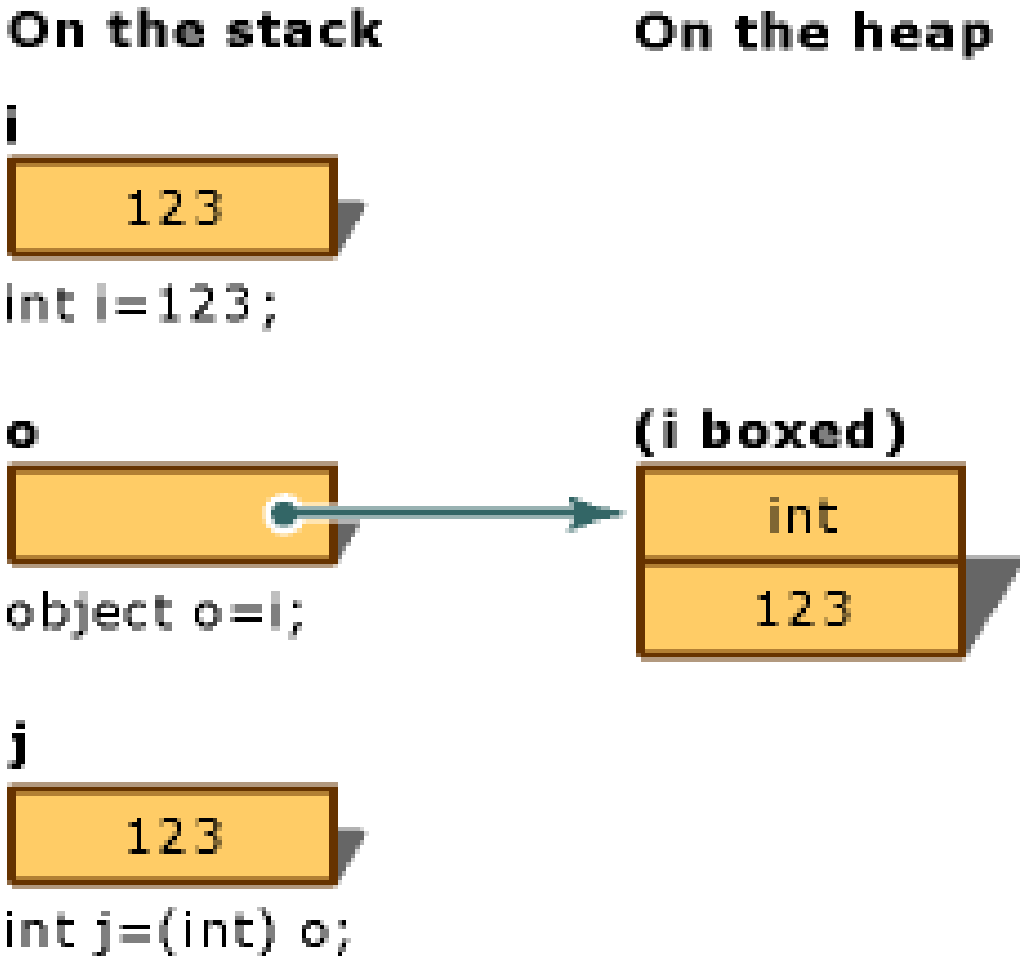
Boxing: To cast Value type to Object Type This process is known as “Boxing”.

```
static void Main(string[] args)
    {
        int i = 30;
        Object MyObj = i;
        Console.WriteLine(MyObj);
        Console.ReadLine();
    }
```



Unboxing: To cast Object type to Value Type This process is known as “Unboxing”.

```
static void Main(string[] args)
    {
        Object MyObj = 123;
        int i = (int)MyObj;
        Console.WriteLine(i);
       Console.ReadLine();
    }
```



C# Coding Standards

- Class and Method names should always be in Pascal Case.
- Method argument and Local variables should always be in Camel Case.
- Avoid the use of underscore while naming identifiers.
- Avoid the use of System data types and prefer using the data types.
- Always prefix an interface with letter I.
- For better code indentation and readability always align the curly braces vertically.
- Always use the using keyword when working with disposable types.
- Always declare the variables as close as possible to their use.
- Constants should always be declared in UPPER_CASE.
- Use camel casing ("camelCasing") when naming private or internal fields, and prefix them with _
- When working with static fields that are private or internal, use the s_ prefix.

References

https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/tutorials/

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/built-in-types

https://dotnet.microsoft.com/en-us/learn/dotnet/architecture-guides

https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/types/casting-and-type-conversions

https://www.tutorialsteacher.com/csharp

https://www.javatpoint.com/c-sharp-tutorial

https://dotnettutorials.net/lesson/type-casting-in-csharp/